

Jenkins: Arquitetura e Pipelines Declarativos

Orquestração de CI/CD, escalabilidade com agentes e automação via código.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Sumário

- Ecossistema Jenkins e CI/CD
- Arquitetura Distribuída: Controlador e Agentes
- Pipeline as Code: Declarativo vs Scripted
- Anatomia do Pipeline Declarativo
- Demonstração e Discussão Prática

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Ecossistema Jenkins e CI/CD

Jenkins é um servidor de automação open-source amplamente utilizado para orquestrar pipelines de **CI/CD** (Integração Contínua e Entrega Contínua) no ciclo de vida de desenvolvimento de software (SDLC). Ele automatiza processos *end-to-end*, desde o *push* do código pelos desenvolvedores até a implantação em produção. Sua arquitetura modular e extensível é suportada por mais de 1.800 plugins, permitindo a integração com diversas ferramentas do ecossistema DevOps, como Git, Docker, Kubernetes e Maven, facilitando a colaboração e a entrega ágil de software.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Arquitetura Distribuída do Jenkins

O Jenkins emprega uma arquitetura de topologia distribuída, conhecida como **Controlador e Agentes** (Controller/Agents), que permite escalabilidade e distribuição de carga de trabalho. O **Controlador** atua como o cérebro central, gerenciando configurações, agendando *builds* e monitorando a segurança. Os **Agentes** são instâncias responsáveis pela execução das tarefas reais, recebendo instruções do controlador. Dentro de cada agente, os **Executores** representam os *slots* que definem a capacidade de processamento paralelo e simultâneo de tarefas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



O Controlador Jenkins: O Cérebro

O **Controller** (antigamente Master) é o coração da instalação do Jenkins, gerenciando plugins, orquestrando builds em agentes, provendo a interface de usuário e API, e monitorando o histórico. Sua função é estritamente de gestão, evitando a execução de tarefas pesadas para prevenir gargalos no sistema.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Os Agentes: Força Tarefa



Isolamento de Ambiente

Os agentes permitem o **isolamento de ambiente**, crucial para gerenciar dependências de projetos variados. Por exemplo, você pode configurar um agente com Java 8 para dar suporte a *projetos legados* e outro com Node.js 18 para novos serviços, garantindo que não haja conflitos de ambiente no servidor principal do Jenkins. Essa capacidade é fundamental para manter a estabilidade e a compatibilidade entre diferentes *pipelines* de construção.



Diversidade de Infraestrutura

A **infraestrutura** que hospeda os agentes é bastante flexível. Eles podem ser provisionados em uma vasta gama de recursos computacionais, incluindo máquinas físicas dedicadas, máquinas virtuais (VMs) hospedadas em ambientes de virtualização, ou até mesmo containers leves, como Docker e orquestrados por plataformas como Kubernetes. Essa diversidade permite que o Jenkins se adapte a qualquer ambiente de TI existente.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Os Agentes: Força Tarefa



Provisionamento Dinâmico

Em arquiteturas de nuvem e *microserviços* modernas, o Jenkins pode empregar o **provisionamento dinâmico** de agentes. Isso significa que os agentes podem ser criados 'sob demanda' (on-demand) em *clusters* Kubernetes no momento em que uma tarefa é iniciada e, subsequentemente, destruídos logo após a conclusão do processo de *build*. Essa estratégia otimiza significativamente os custos de nuvem, pois os recursos são consumidos apenas quando estritamente necessários.



Comunicação Eficiente

A **comunicação** entre o agente e o controlador Jenkins é estabelecida por meio de protocolos robustos e seguros. Os métodos comuns incluem JNLP (Java Network Launch Protocol), que permite que o agente se conecte ao controlador e receba comandos, ou SSH (Secure Shell), que oferece uma conexão segura para execução remota de comandos. Essa comunicação eficiente garante a coordenação e o fluxo de trabalho contínuo das tarefas.

INSTITUIÇÃO EXECUTORA



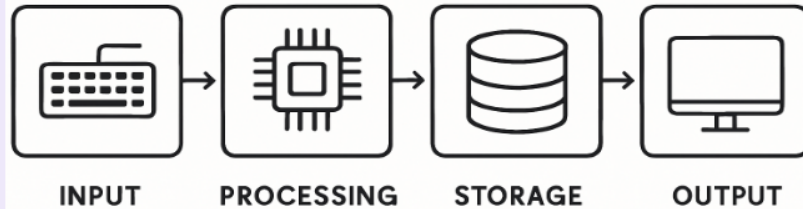
COORDENADORA



APOIO



Pipeline as Code



Fluxo de dados: entrada, processamento, armazenamento e saída.

- O conceito de **Pipeline as Code** representa uma evolução no Jenkins, onde a configuração dos processos de entrega é definida em arquivos de texto em vez de através da interface gráfica tradicional.
- O **Versionamento** é uma vantagem crucial, pois o arquivo do pipeline (Jenkinsfile) é armazenado no mesmo repositório do código-fonte (Git), permitindo que todas as mudanças sejam rastreadas via commits.
- Com o **Code Review**, qualquer alteração na forma como o software é construído, testado e implantado passa pelo mesmo rigoroso processo de revisão que o próprio código da aplicação.

INSTITUIÇÃO EXECUTORA



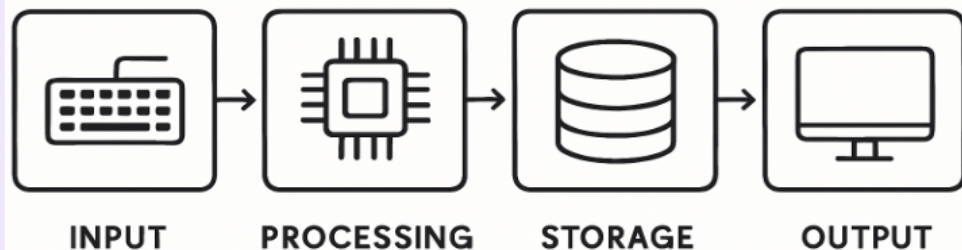
COORDENADORA



APOIO



Pipeline as Code



Fluxo de dados: entrada, processamento, armazenamento e

- A **Resiliência** do sistema é aprimorada significativamente; caso o servidor Jenkins principal seja perdido, o pipeline pode ser rapidamente reconstruído a partir do arquivo versionado, minimizando o tempo de inatividade.
- A **Auditabilidade** é garantida, fornecendo um histórico claro e detalhado de quem realizou alterações no fluxo de CI/CD e quais foram os motivos por trás dessas modificações.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Pipelines: Declarativo vs. Scripted

Esta tabela detalha as diferenças fundamentais entre os pipelines Declarativo e Scripted no Jenkins, auxiliando na escolha do modelo mais adequado.

Característica	Pipeline Declarativo	Pipeline Scripted
Sintaxe	Estruturada e rígida (Opinativa)	Baseada em Groovy (Flexível)
Curva de Aprendizado	Baixa (mais legível)	Alta (exige conhecimentos de programação)
Tratamento de Erros	Blocos pré-definidos (post)	Try-Catch tradicionais
Validação	Verificação sintática antes da execução	Erros podem aparecer apenas no runtime

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Estrutura do Pipeline Declarativo

1

Bloco Raiz 'pipeline'

Este é o bloco fundamental que encapsula toda a definição do seu Pipeline Declarativo. Ele serve como o invólucro principal para todas as configurações e estágios do processo de CI/CD.

2

Definir 'agent'

O bloco 'agent' é crucial para especificar onde o pipeline, ou um estágio específico, será executado. Ele pode definir o executor em um nó específico, em um container Docker, ou de forma mais genérica, determinando o ambiente de execução.

3

Container 'stages'

Este bloco funciona como um container para agrupar todas as fases lógicas do seu pipeline. Dentro de 'stages', você definirá a sequência de 'stage' individuais que compõem o fluxo de trabalho completo.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Estrutura do Pipeline Declarativo

4

Definir 'stage'

Cada 'stage' representa uma fase distinta no ciclo de vida do pipeline, como 'Compilação', 'Teste' ou 'Deploy'. É uma unidade lógica de trabalho que agrupa um conjunto de 'steps' relacionados para uma finalidade específica.

5

Comandos em 'steps'

Dentro de cada 'stage', o bloco 'steps' contém os comandos e instruções específicos a serem executados. Estes são os passos atômicos, como a execução de scripts shell, invocação de ferramentas ou chamadas para funções do Jenkins, que realizam o trabalho daquele estágio.

6

Ações 'post'

O bloco 'post' permite definir ações que serão executadas após a conclusão do pipeline ou de um estágio específico, independentemente do seu resultado. Pode ser configurado para rodar em casos de sucesso, falha, ou sempre, para tarefas como limpeza, notificações ou arquivamento de artefatos.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Bloco 'agent'

O bloco ``agent`` é **crucial** para definir o ambiente onde todo o pipeline ou um estágio específico será executado. Se especificado como ``agent any``, o Jenkins utilizará qualquer executor (agente) disponível na sua *pool* para iniciar a execução. Alternativamente, pode-se especificar um agente com base em sua *label* (rótulo) ou capacidades, por exemplo, ``agent { label 'linux' }``. Quando o bloco ``agent`` é definido dentro de um ``stage`` particular, ele **sobrescreve** a configuração de agente global do pipeline, permitindo que aquele estágio rode em um ambiente de execução diferente, como um estágio de compilação em um agente Linux e os testes em um agente Windows.

Blocos 'stages' e 'stage'

O bloco ``stages`` funciona como um **container** para a lógica principal do pipeline, agrupando uma série de blocos ``stage`` individuais. Cada ``stage`` representa uma fase distinta e lógica do processo de CI/CD (Integração Contínua/Entrega Contínua), como 'Build', 'Test' ou 'Deploy'. É fundamental que cada ``stage`` tenha um **nome descritivo**, pois este nome será exibido na interface visual do Jenkins, como o Blue Ocean. Essa visibilidade facilita enormemente a depuração, permitindo identificar rapidamente em qual fase específica (e.g., 'Compilação', 'Testes Unitários' ou 'Deploy') um erro ocorreu, melhorando a *rastreabilidade* e a resolução de problemas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Bloco `steps`

O bloco `steps` (passos) é fundamental e reside dentro de um `stage` (estágio) em um pipeline declarativo do Jenkins. Ele define a sequência de comandos e ações reais que serão executadas durante aquele estágio específico. É aqui que são chamadas ferramentas externas, como **Maven**, **Gradle**, ou **NPM**, bem como scripts customizados, utilizando comandos como `sh 'make'` para scripts shell, `bat 'run.bat'` para comandos do Windows, ou `echo 'Finalizado'` para mensagens simples. Este bloco representa o cerne da lógica de execução de cada parte do seu processo de CI/CD.

Bloco `post`

O bloco `post` (pós-execução) permite definir ações a serem realizadas ao final de um `stage` ou de todo o pipeline, independentemente do seu resultado primário. Ele é ideal para tarefas de notificação e limpeza. Diferentes condições podem ser especificadas para a execução de sub-blocos dentro do `post`: `always` executa sempre; `success` apenas se o estágio/pipeline foi bem-sucedido; `failure` se falhou; e `cleanup` para tarefas de limpeza do workspace, como apagar arquivos temporários. Isso garante que ações importantes sejam tomadas, como enviar mensagens de status ou liberar recursos, mesmo em caso de falha.



INSTITUIÇÃO EXECUTORA



COORDENADORA



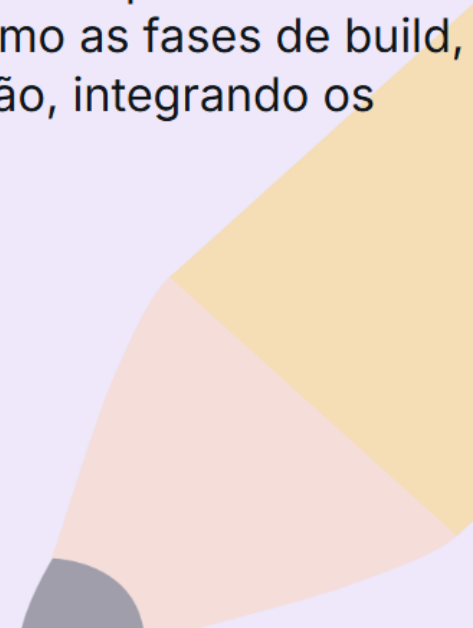
APOIO





Demonstração Guiada

Demonstraremos a anatomia de um **Jenkinsfile** real, um script declarativo que orquestra um pipeline CI/CD para um projeto Java/Maven. Esta demonstração ilustrará como as fases de build, teste e deploy são definidas e como o bloco 'post' é utilizado para notificação, integrando os conceitos teóricos de 'steps' e 'post' vistos anteriormente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Demonstração Guiada

O pipeline é iniciado com a declaração `pipeline { agent any }`. Esta configuração é crucial, pois informa ao Jenkins que o

- 1 pipeline está sendo definido de forma declarativa e especifica que qualquer agente (nó de execução) disponível no ambiente Jenkins pode ser utilizado para executar as etapas subsequentes, garantindo flexibilidade na execução.

A fase de 'Build' é definida no estágio 'Build'. Dentro deste estágio, um comando `echo 'Iniciando o build...'` é usado para sinalizar o começo desta fase. Em um Jenkinsfile real, este seria o ponto onde ferramentas de build como Maven ou Gradle seriam invocadas para compilar o código-fonte, empacotar artefatos e preparar o ambiente para as próximas etapas.

- 2
- 3 O estágio 'Test' é dedicado à verificação de qualidade do código. Aqui, comandos como `sh 'mvn test'` seriam executados para rodar testes unitários e de integração. Este passo é fundamental para garantir que o código recém-construído esteja funcional e livre de regressões antes de prosseguir para o deploy.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Demonstração Guiada

4

A fase de 'Deploy' é representada no estágio 'Deploy'. Nesta etapa, operações como o push de uma imagem Docker para um registro de contêineres (``sh 'docker push my-app:latest'``) ou a implantação de artefatos em um ambiente de homologação ou produção seriam realizadas. Este é o passo final na entrega contínua, onde o software é disponibilizado para uso.

5

O bloco ``post`` é utilizado para ações que ocorrem após a conclusão do pipeline, independentemente do seu sucesso ou falha. Ele garante que o desenvolvedor seja notificado do status final da execução via logs do console (ex: ``always { echo 'Pipeline finalizado!' }`` ou ``success { echo 'Pipeline executado com sucesso!' }``). Isso é vital para feedback imediato e para iniciar ações de acompanhamento, como limpeza de ambiente ou envio de relatórios.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Exemplo Prático de Jenkinsfile Declarativo

- A estrutura **pipeline** encapsula a definição completa do pipeline, servindo como o bloco raiz para todos os elementos subsequentes do script.
- A diretiva **agent any** instrui o Jenkins a executar este pipeline em qualquer agente disponível, gerenciado pelo controlador Jenkins, garantindo flexibilidade na alocação de recursos.
- Dentro do bloco `pipeline`, as **stages** (estágios) organizam o trabalho em seções lógicas e sequenciais, como Build, Test e Deploy, permitindo uma visualização clara do fluxo de CI/CD (Integração Contínua/Entrega Contínua).
- Cada estágio contém um bloco **steps** (passos) que define as ações a serem executadas, neste exemplo, utilizando o comando `echo` para simular as operações de compilação, teste e implantação.
- A seção **post** é um bloco opcional que define ações a serem realizadas após a conclusão do pipeline, independentemente do resultado, permitindo a execução de passos específicos para sucesso (`success`) ou falha (`failure`).

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



groovy

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Construindo o projeto...'
                // Exemplo: sh 'mvn clean install'
            }
        }

        stage('Test') {
            steps {
                echo 'Executando os testes...'
                // Exemplo: sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Realizando o deploy...'
                // Exemplo: sh 'docker push minha-imagem'
            }
        }
    }

    post {
        success {
            echo 'Pipeline executado com sucesso!'
        }
        failure {
            echo 'O pipeline falhou. Verifique os erros.'
        }
    }
}
```

INSTITUIÇÃO EXECUTORA

COORDENADORA

APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Qual é a função do bloco 'agent' em um pipeline declarativo do Jenkins?

- A Monitorar o status de saúde do servidor Jenkins em tempo real.
- B Especificar onde e como o pipeline (ou um estágio) será executado.
- C Definir quem é o usuário responsável por aprovar o deploy.
- D Listar todos os plugins que precisam ser instalados no controlador.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Qual é a função do bloco 'agent' em um pipeline declarativo do Jenkins?

- ✗ Monitorar o status de saúde do servidor Jenkins em tempo real.
- ✓ Especificar onde e como o pipeline (ou um estágio) será executado.
- ✗ Definir quem é o usuário responsável por aprovar o deploy.
- ✗ Listar todos os plugins que precisam ser instalados no controlador.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Discuta com o colega ao lado

Considerando uma empresa que realiza 500 builds por dia em diferentes tecnologias (Python, Java e Go), como a arquitetura de Agentes Distribuídos resolve o problema de concorrência e conflito de dependências?



Refleta sobre como evitar gargalos de recursos e isolar ambientes de execução para garantir eficiência.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Conclusão

- Jenkins é o orquestrador líder de CI/CD, essencial para automação de ponta a ponta no SDLC, com vasta extensibilidade via mais de 1.800 plugins.
- A arquitetura de Controlador e Agentes garante escalabilidade, permitindo execução isolada de tarefas em múltiplos ambientes, incluindo contêineres e VMs.
- Pipelines Declarativos, versionados como código (Jenkinsfile), promovem auditabilidade e resiliência, substituindo configurações manuais por descrições legíveis.
- O Jenkinsfile estrutura a execução em stages e steps, e o bloco 'post' gerencia ações pós-execução, como notificações de sucesso ou falha.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

